

Tenxo

GPU Compute on GitHub Infrastructure — Progress, Roadmap & Competitive Landscape

"Your code stays encrypted from your machine to the result. Nobody in between can read it."

What We're Building

Tenxo is a GPU compute platform where users can run ML training, rendering, or data processing jobs on remote GPUs without ever exposing their code or data to the platform itself. Everything is encrypted on the user's machine before upload. The remote machine decrypts only in memory, runs the job in an isolated container, re-encrypts the results, and sends them back. The coordination server never sees any plaintext data, never holds encryption keys, and cannot inspect what's being run.

Where We Are Today

Component	Status	Notes
Client-side encryption	Done	User encrypts their workspace ZIP with AES-256-GCM before upload. The key is split between user and runner via cryptographic handshake — server never sees it.
Runner registration	Done	GPU-equipped machines (edge nodes) report in every 20 seconds with their GPU type, VRAM, and status. Users see available nodes.
Job assignment & key exchange	Done	Matchmaker routes jobs to available runners. Runner and user agree on a session key through the matchmaker without revealing it to the matchmaker.
Docker execution	Done	Runner unpacks the encrypted workspace into a Docker container, executes the job, captures output. Supports Python, CUDA, Blender, and custom workloads.
Result encryption & retrieval	Done	Runner re-encrypts the result and uploads it. User downloads and decrypts locally. Integrity hashes verify the job ran the correct input.
GPU detection	Done	Supports NVIDIA (nvidia-smi) and AMD (rocm-smi). GPU model and memory reported on registration.
Provider identity	In progress	Runners currently identify with a random ID. No cryptographic proof during check-in.
Storage layer	In progress	Currently uses local disk. Being migrated to GitHub Releases as the storage backend — eliminates need for separate cloud storage.
Docker Registry push	In progress	Users will push their workspace as a private Docker image. Runner pulls the tagged image from the registry, matching the existing matchmaker flow.
Web dashboard	Not started	Basic job submission via CLI only. No visual interface or real-time monitoring yet.
Economic model	Not started	No pricing, staking, or payment mechanism yet. Jobs run on trusted test nodes.

What We're Fixing Next

1. Provider Identity — How Nodes Prove They're Real

Problem: Runners check in with a random ID that anyone could fake. A malicious actor could register fake nodes and flood the marketplace.

Solution: On first boot, each runner generates a permanent key pair (like a digital ID card). Every check-in includes a signature proving the runner holds the matching key. The server stores the public key on first registration and rejects bad signatures. Same concept as Akash (Cosmos wallet) and io.net (Solana wallet), but simpler — no blockchain needed. The tradeoff: blockchain wallets give portable identity backed by on-chain reputation; our approach is lighter but doesn't benefit from blockchain guarantees.

2. Central Coordinator — Single Point of Failure

Problem: One matchmaker server runs the whole marketplace. If it goes down, everything stops.

Solution: Near-term: run multiple matchmaker instances behind a load balancer with shared state (NATS). Long-term: make the matchmaker fully stateless. A blockchain-level approach (Akash model) is possible later but adds major complexity.

3. Economic Guarantees — Trust Without Penalties

Problem: A runner can take a job, disappear mid-execution, and face no consequences. No penalties, no reputation.

Solution: Start with a reputation score per runner (completion rate, uptime, response time) visible to users. Later, add staking — runners deposit collateral that gets slashed if they fail jobs. Mirrors io.net's Solana staking and Akash's escrow. Start off-chain, graduate to on-chain when needed.

4. Storage & Code Delivery — Moving Beyond R2

Problem: Single-provider storage (R2, S3) creates a dependency. If that provider blocks us or goes down, all payloads are stranded. Users also need flexibility in how they ship code to runners.

Solution: Three complementary options:

- **GitHub Releases (primary):** Each job gets its own Release: user uploads encrypted payload, runner downloads it, executes, re-encrypts result, uploads it back as a new asset on the same Release. User pulls from GitHub. Everything stays in the GitHub ecosystem — no separate storage provider. Latency is 100–500ms via Azure CDN. Main limits: 2GB per asset, 5000 API req/hr.
- **Docker Registry (code delivery):** Users push their workspace as a private Docker image to any OCI-compatible registry (Docker Hub, GitHub Container Registry, self-hosted). The runner pulls the image by tag directly — no file upload/download step, no encryption layer needed for transport. The matchmaker passes the image tag to the runner as part of the job assignment. This is the natural fit for container-native workloads and matches io.net's model. Limits: 10GB per layer on Docker Hub, higher on GHCR; pull rate limits apply (100–500 pulls/6hr on Docker Hub free tier).
- **Storj (fallback):** S3-compatible decentralized storage for larger payloads that exceed GitHub's 2GB per-asset cap. ~\$80/mo for 10TB.

Storage Option	Retrieval Speed	Decentralized?	Cost (10TB + 5TB egress)	Key Tradeoff
GitHub Releases	100–500ms (Azure CDN)	No	Free (within limits)	2GB per-asset cap, API rate limits
Docker Registry	50–200ms (CDN cached)	No	Free tier (Docker Hub) / GHCR	Best for container workloads; pull rate limits; no encryption layer needed
Cloudflare R2	50–150ms (global edge)	No	\$158/mo	Zero egress, but single provider
Storj	100–500ms (parallel downloads)	Yes	~\$80/mo	S3-compatible, decentralized, slower peak
IPFS + Pinata	60ms–5s (depends on caching)	Yes	Varies	Content-addressed but first retrieval slow
AWS S3	10–50ms (same region)	No	\$689/mo	Fastest but most expensive, vendor lock-in
Backblaze B2 + CF	50–200ms	No	\$110/mo	Cheapest centralized, free egress via CF

Our pick: GitHub Releases first (no extra infra), Docker Registry for container-native jobs, Storj as fallback for larger payloads. All three integrate with the matchmaker — the job assignment carries either an asset URL or an image tag.

Competitor Landscape

Dimension	Tenxo (Today)	Tenxo (Planned)	Akash Network	io.net
Privacy	Strongest — server never sees your data.	Same + traffic obfuscation.	Code visible to provider. No built-in encryption.	Data encrypted, provider can't read it.
Provider Identity	Weak — runners use random IDs.	Cryptographic keys on each runner. Server verifies signatures.	Strong — providers identified on blockchain. Bad behavior costs them.	Strong — Solana wallet identity + reputation scores.
Storage & Code Delivery	Migrating to GitHub Releases + Docker Registry.	GitHub Releases + Docker Registry + Storj fallback.	IPFS-based. Decentralized but slower.	AWS S3. Fast but centralized.
GPU Availability	NVIDIA + AMD. One default image per job type.	ROCm images for AMD. Custom images from any registry.	NVIDIA only. Full Docker registry support.	NVIDIA only. Rich image selection, CUDA 11/12.
Decentralization	Low — one server coordinates everything.	Multiple servers with shared state. Stateless design.	High — runs on its own blockchain.	Medium — Solana for identity, but job scheduling is centralized.
Economics	None — no pricing or penalties.	Reputation scores + optional staking.	AKT token. Escrow leases. Provider slashing.	Solana staking. Hourly billing. Reputation system.
User Experience	Simple CLI, one command. No dashboard.	Web dashboard + job monitoring.	Complex config files. Steep learning curve.	Polished dashboard + CLI. Fastest onboarding.

Bottom Line. Tenxo already has the strongest privacy model in this space — your data is encrypted end-to-end and the coordinator never sees it. What we lack today is the economic and identity infrastructure that more mature platforms have built. The next milestones are: cryptographic runner identity (keypair-based signatures on check-ins), GitHub Releases + Docker Registry as storage & code delivery, a reputation system for runners, and a simple web dashboard. Each of these is independently valuable and builds toward a competitive, production-ready platform.